Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 233 (2024) 604–613

Procedia Computer Science

www.elsevier.com/locate/procedia

5th International Conference on Innovative Data Communication Technologies and Application (ICIDCA 2024)

Optimizing Task Scheduling in Cloud Computing: An Enhanced Shortest Job First Algorithm

Yellamma Pachipala[a], Kavya Sri Sureddy[b], A.B.S. Sriya Kaitepalli[c], Nagalakshmi Pagadala[d], Sai Satwik Nalabothu[e], Mihir Iniganti[f]

[a,b,c,d,e,f] *Dept of CSE, Koneru Lakshmaiah Education Foundation,Vaddeswaram, India*

Abstract

In the dynamic landscape of cloud computing, efficient task scheduling plays a pivotal role in optimizing resource utilization and enhancing overall system performance. This research introduces a groundbreaking approach to task scheduling in cloud environments through the implementation of a novel Modified Shortest Job First (SJF) algorithm within the CloudSim simulation framework. The primary objectives of this study are to address existing challenges in traditional scheduling algorithms, mitigate resource bottlenecks, reduce task completion times, and improve overall system efficiency.

© 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application

Keywords: Cloud Computing; CloudSim; Efficiency; Job Scheduling; Modified Shortest Job First (SJF); Optimization; Resource Allocation; Resource Bottlenecks; Simulation Framework; System Performance; Task Completion Times; Task Scheduling;

1. Introduction

Task scheduling is a critical aspect of cloud computing, influencing the effective utilization of resources and the overall performance of cloud-based applications. Traditional scheduling algorithms face challenges in adapting to the dynamic and heterogeneous nature of cloud workloads. Motivated by the need for enhanced efficiency and responsiveness, this research explores the adoption of the SJF algorithm and introduces modifications tailored to the cloud environment within the CloudSim simulation framework [3]. Cloud computing environments are characterized by varying workloads, resource heterogeneity, and dynamic user demands [4]. Traditional scheduling algorithms often struggle to adapt to these complexities, resulting in suboptimal resource utilization and increased task completion times [5]. The SJF algorithm, known for its simplicity and efficiency in traditional computing environments, serves as a foundation for our proposed modifications.

The motivation behind adopting the SJF algorithm lies in its ability to prioritize tasks based on their execution time, theoretically minimizing the time required to complete a set of tasks [6]. However, to optimize its application in cloud

1877-0509 © 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application
10.1016/j.procs.2024.03.250

computing, our research introduces novel features to address specific challenges. These features include dynamic prioritization, predictive load balancing, elastic resource allocation, priority aging, adaptive scheduling intervals, and task pre-emption [8]. Dynamic prioritization considers both task length and current resource availability, allowing the algorithm to adapt to varying workloads effectively [9]. Predictive load balancing leverages historical data and analytics to forecast resource demands, proactively mitigating potential bottlenecks [14]. Elastic resource allocation enables dynamic scaling of resources based on task requirements, ensuring optimal allocation throughout task execution.

In the subsequent sections, we delve into the details of our proposed SJF algorithm, its implementation within the CloudSim framework, and the outcomes of comprehensive evaluations [15]. The results of our experiments highlight the algorithm's effectiveness in improving resource utilization, reducing task completion times, and promoting fairness in resource allocation [16]. Through this research, we aim to make a significant contribution to the field of cloud task scheduling, providing insights that pave the way for more efficient and adaptive cloud computing environments[17].

The paper's organization of the information follows: Section 2 offers a comprehensive literature review of existing research on Task Scheduling algorithms. In Section 3, we discuss the current methodology for optimization of resource allocation and scheduling and introduce our proposed methodology (Improvised of SJF) with priority aging in cloud computing environments. We explain the task scheduling process in detail. Section 4 presents the results and comparative analysis of our proposed methodology. Finally, Section 5 provides a detailed explanation of the outcomes and the ending statements of the research paper.

2. Related Work

Task scheduling in cloud computing has been a subject of extensive research, with various algorithms proposed to optimize resource utilization and enhance overall system performance [18][19]. Traditional scheduling algorithms, such as Round Robin and First-Come-First Serve, have been widely used but face challenges in adapting to the dynamic and heterogeneous nature of cloud workloads. Table1 below shows the literature review on various methodologies.

2.1. Identified Gaps in the Literature

In the existing literature, there is a discernible gap in addressing the dynamic and unpredictable nature of cloud workloads using SJF algorithms [20]. While traditional SJF algorithms prioritize tasks based solely on length, they may not consider real-time resource availability, historical workload data, or preemptive mechanisms to handle varying priorities [21].

Table 2 below shows the need for a more adaptive and responsive scheduling algorithm that considers both task characteristics and current system conditions and has been identified as a crucial gap in the literature. The proposed SJF algorithm aims to fill this gap by introducing dynamic prioritization, predictive load balancing, elastic resource allocation, priority aging, adaptive scheduling intervals, and task preemption.

2.2. Proposed Algorithm's Contribution:

Our proposed SJF algorithm addresses the identified gaps by incorporating dynamic prioritization, which adapts to both task length and real-time resource availability. Predictive load balancing mitigates potential resource bottlenecks by leveraging historical data and analytics. Elastic resource allocation ensures optimal resource usage by dynamically scaling resources based on task requirements. The inclusion of priority aging prevents task starvation, promoting fairness in resource allocation. Adaptive scheduling intervals improve responsiveness by adjusting scheduling frequencies according to the system's current load.

Task preemption allows higher-priority tasks to preempt lower-priority tasks, enhancing overall system efficiency. Through these innovative features, the proposed SJF algorithm aims to contribute to the literature by providing a more adaptable, efficient, and fair task scheduling mechanism tailored to the dynamic nature of cloud computing environments. The algorithm's design fills crucial gaps identified in existing literature, making a significant stride toward addressing the challenges of cloud task scheduling.

Table 1. Literature Review on Existing methodology

S.NO	Title	Authors	Applied Methodology	Drawbacks
1.	A Comparative Study of Task Scheduling Algorithms in Cloud Computing	Sandeep Kumar, Rakesh Kumar, and Rajesh Kumar [2]	Employing a comparative study, the paper scrutinizes various task scheduling algorithms in cloud computing. Additionally, it introduces a novel hybrid genetic algorithm (HGA) that amalgamates genetic algorithm features with the min-min algorithm.	The paper's drawbacks are that it does not provide a theoretical analysis of the complexity and convergence of the HGA.
2.	Task Scheduling and Resource Allocation in Cloud Computing Using a Heuristic Approach	Xiaoyu Liang, Mahendra Bhatu Gawali, and Subhash K. Shinde[22]	Utilizing a heuristic approach, the paper tackles task scheduling and resource allocation in cloud computing. This method integrates modified analytic hierarchy process (MAHP), bandwidth-aware divisible scheduling (BATS) + BAR optimization, longest expected processing time pre-emption (LEPT), and divide-and-conquer strategies.	The paper lacks a theoretical analysis of the heuristic approach's complexity and convergence.
3.	STASR: A New Task Scheduling Algorithm for Cloud Environment	N. Zanoon and D. Rawshdeh[13]	Conducting a comparative study, the paper assesses diverse task scheduling algorithms in cloud computing. It also presents the STASR algorithm, which operates on the distribution of tasks and resources within groups, employing parameters and standards to diminish waiting times and enhance responses.	The paper's drawbacks are that It lacks a theoretical analysis of the STASR algorithm's complexity and convergence.
4.	Research and Simulation of Task Scheduling Algorithm in Cloud Computing	Xiangmo Zhao, Xiaoyu Liang, and Zhenyu Liu [12]	The paper uses a simulation study as the applied methodology to evaluate the performance of different task scheduling algorithms in cloud computing. The paper also proposes a new task scheduling algorithm, called the improved genetic algorithm (IGA), which improves the genetic algorithm by adding a mutation operator and a crossover operator.	The paper's drawbacks are that it does not provide a theoretical analysis of the complexity and convergence of the IGA algorithm
5.	Cloud Computing CPU Allocation and Scheduling Algorithms Using CloudSim Simulator.	Gibbet Tani Hicham and El Amrani Chaker [11]	Through a simulation study, the paper assesses the performance of different CPU allocation and scheduling algorithms in cloud computing. Introducing the improved round robin (IRR) and improved first come first served (IFCFS) algorithms, the paper enhances existing approaches by considering CPU utilization and task waiting times.	The paper suffers from the following drawback in its research: It overlooks the effect of communication cost and resource reliability on the cloud environment, which may alter the CPU allocation and scheduling performance.
6.	A Novel Task Scheduling Algorithm Based on Improved Ant Colony Optimization in Cloud Computing	Yan Zhang, Xiaoyu Song, and Yuhua Qian.[1]	Utilizing a simulation study, the paper appraises diverse task scheduling algorithms in cloud computing. It introduces the improved ant colony optimization (IACO) algorithm, refining the ant colony optimization approach by integrating a pheromone updating strategy and a heuristic function.	The paper's drawbacks are that it does not provide a theoretical analysis of the complexity and convergence of the IACO algorithm.
7.	Task Scheduling in Cloud Computing.	Mir Salim Ul Islam and Bhawana Rana.[10]	The paper uses a comparative study as the applied methodology to analyze the different task scheduling algorithms in cloud computing. The paper reviews the existing literature on task scheduling algorithms and identifies the advantages and disadvantages of each algorithm.	The paper's drawbacks are that It neglects the impact of communication cost and resource reliability on the cloud environment, which may affect the task scheduling quality and efficiency.
8.	A Study of Available Process Scheduling Algorithms and Their Improved Substitutes	Adwait Patil, Sameer Mahajan, and Nahush Kulkarni.[25]	The paper's applied methodology is to study the fundamentals of process scheduling in Real-Time and Distributed systems, and to discuss the algorithms currently being used by various operating systems, such as First-Come-First-Serve, Highest Response Ratio Next, Multilevel Feedback, O (1) Scheduler, and Round Robin. The paper also examines their improved substitutes, such as Genetic Algorithm, Ant Colony Optimization, Particle Swarm Optimization, and Artificial Neural Network. The paper uses mathematical models, graph-theory based methods, and heuristic methods to analyze the performance and efficiency of the algorithms.	The paper's drawbacks are that it does not provide a theoretical analysis of the complexity and convergence of the heuristic approach, and that it does not compare the results of the improved substitutes with the existing algorithms. The paper also does not address the challenges of process scheduling in heterogeneous and dynamic environments, such as cloud computing and edge computing.

Table 2. Drawbacks covered by Proposed SJF Algorithm.

SJF Algorithm	Proposed SJF Algorithm
Lack of Real-time Resource Consideration in Task Prioritization	Dynamic Prioritization: Considers both task length and real-time resource availability
Limited Predictive Mechanisms for Load Balancing	Predictive Load Balancing: Leverages historical data and analytics for proactive load balancing, preventing potential resource bottlenecks
Inflexible Resource Allocation Assumptions	Elastic Resource Allocation: Dynamically scales resources based on task requirements, ensuring optimal allocation throughout task execution.
Risk of Task Starvation Due to Fixed Priorities	Priority Aging: Prevents task starvation by gradually increasing task priorities over time, promoting fairness in resource allocation
Insufficient Responsiveness to Dynamic Workloads	Adaptive Scheduling Intervals: Adjusts scheduling frequencies based on the current system load, improving responsiveness to dynamic workloads
Lack of Task Pre-emption Mechanisms	ask Pre-emption: Allows higher-priority tasks to pre-empt lower-priority tasks, enhancing overall system efficiency

3. Proposed Work

The Modified SJF algorithm was seamlessly integrated into the widely used CloudSim simulation framework for cloud computing environments. Implemented as an extension of CloudSim's scheduling policies, the algorithm allowed for a direct comparison with traditional scheduling algorithms. Various parameters, such as task lengths and resource capacities, were configured to simulate diverse cloud workload scenarios. The simulation environment featured a realistic data center architecture with multiple hosts and a network, aiming to represent a scalable and heterogeneous cloud infrastructure. Diverse workloads, including tasks of varying durations and resource requirements, were generated to assess the algorithm's adaptability under dynamic conditions. The resource configurations reflected the heterogeneity observed in actual cloud environments and were allocated based on the proposed elastic resource allocation mechanism.

3.1. Proposed the Enhanced Short Job First (SJF) Algorithm:

Algorithm Overview: In response to the challenges posed by traditional SJF algorithms in cloud computing environments, our proposed modification integrates a set of innovative features to enhance adaptability, responsiveness, and overall efficiency in task scheduling. The algorithm is designed to consider both task characteristics and real-time system conditions, ensuring optimal resource utilization and improved task completion times.

Unique Features and Improvements:
Dynamic Prioritization: Its objective is to adapt task prioritization based on both task length and real-time resource availability.
Explanation: The algorithm dynamically adjusts task priorities, considering the length of the task and the current availability of resources. This allows for better adaptation to varying workloads and resource conditions.

$$Priority(task) = BasePriority(task) + f(TaskLength(task), ResourceAvailability(resources)). \quad (1)$$

Elastic Resource Allocation: Its objective is enabling dynamic scaling of resources based on task requirements, ensuring optimal allocation throughout task execution.
Explanation: The algorithm supports elastic resource allocation, allowing resources to dynamically scale based on the requirements of executing tasks. This ensures efficient and optimal resource utilization during varying workloads.
Priority Aging: Its objective is to prevent task starvation by gradually increasing the priority of tasks over time.
Explanation: The algorithm introduces priority aging, a mechanism that ensures tasks do not remain in a low-priority state indefinitely. Gradually increasing task priorities over time promotes fairness in resource allocation and prevents the risk of task starvation.

$$Priority(task, t) = BasePriority(task) + AgingFactor \times t \quad (2)$$

Adaptive Scheduling Intervals: Its objective is to adjust the scheduling interval based on the current system load,

improving responsiveness.
Explanation: The algorithm dynamically adapts scheduling intervals according to the current system load. Shorter intervals are employed during high loads, enhancing responsiveness, while longer intervals are used during low loads to conserve resources.
Task Preemption: Its objective is to allow higher-priority tasks to preempt lower-priority tasks, enhancing overall system efficiency.
Explanation: Task preemption is introduced, enabling higher-priority tasks to preempt the execution of lower-priority tasks when necessary. This contributes to improved system efficiency and ensures that critical tasks receive timely execution.

3.2. Executing Process:

Step 1: Set Up Development Environment:
Initiate a new Java project within your preferred Integrated Development Environment (IDE) like Eclipse or IntelliJ IDEA. Alternatively, employ a text editor along with command-line tools for the development process.
Step 2: Import CloudSim Libraries:
Download the CloudSim library (JAR file) from the official CloudSim website or repository, such as GitHub. Integrate the CloudSim JAR file into your project's build path to ensure seamless accessibility to CloudSim classes and functions.
Step 3: Create VM and Cloudlet Classes:
Construct a class to represent Virtual Machines (VMs) (e.g., ProposedVm) encompassing attributes like MIPS (Million Instructions Per Second), RAM (Memory), storage, and other pertinent parameters.Develop a class to represent cloudlets (e.g., Proposed Cloudlet) with attributes such as length (execution time), required resources (e.g., MIPS, RAM), and any additional characteristics relevant to your simulation.
Step 4: Create a CloudSim Simulation Environment:
Instantiate a CloudSim simulation object to establish a simulation environment tailored for cloud-based resource management. Configure the simulation environment by specifying parameters such as simulation duration, seed for randomization, and other settings crucial to your simulation. Construct a data center within the simulation environment, delineating its architecture and allocating resources. The data center may consist of host machines, data stores, and networking components.
Step 5: Create a List of VMs and Cloudlets:
Generate a comprehensive list of VMs and cloudlets intended for scheduling using the proposed SJF algorithm. Populate these VM and cloudlet objects with attributes, including but not limited to MIPS, RAM, storage, cloudlet length, and resource requirements. It is represented in the fig 1.
Step 6: Implement Proposed SJF Scheduling
Enhance the SJF scheduling by considering additional parameters or constraints pertinent to the proposed algorithm. Sort the list of cloudlets based on their execution time (length) in ascending order, arranging them from the shortest to the longest. Iterate through the sorted list of cloudlets and allocate each cloudlet to an available VM with the least remaining execution time. Keep meticulous track of VM availability and cloudlet allocation decisions.
Step 7: Execute the Simulation
Execute the CloudSim simulation to initiate the scheduling of cloudlets on the selected VMs according to the proposed SJF algorithm. Monitor the simulation's progression, recording relevant data such as task completion times, VM utilization, and scheduling decisions.
Step 8: Refinement and Experimentation
Fine-tune your simulation by adjusting VM and cloudlet configurations or experimenting with variations of the proposed SJF algorithm. Evaluate the impact of these adjustments on overall system efficiency and performance. This structured methodology delineates the steps for implementing the proposed SJF scheduling algorithm in CloudSim, providing a systematic framework for designing, simulating, and analysing scheduling algorithms in cloud computing environments.

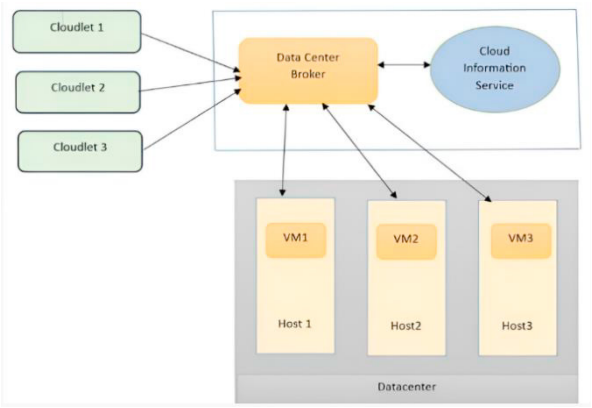


Fig. 1. CloudSim Environment

Sorting Tasks: Initially, tasks are sorted based on their lengths, allowing the algorithm to prioritize shorter tasks for execution.

Task Allocation: The algorithm iteratively selects tasks and attempts to allocate them to available resources. If resources are unavailable, the algorithm waits for availability or pre-emption events.

Dynamic Prioritization: The find Available Resources function dynamically prioritizes resources based on both task length and current availability.

Priority Aging: Task priorities are updated over time to prevent task starvation and promote fairness in resource allocation.

Resource Status Update: The update Resource Status function ensures the availability status of resources is continually updated based on task execution.

The proposed Modified Shortest Job First (SJF) algorithm introduces several key enhancements compared to traditional SJF. It dynamically prioritizes tasks based on both length and real-time resource availability, allowing for adaptability to varying workloads. Predictive load balancing leverages historical data, ensuring proactive resource allocation. Elastic resource allocation dynamically scales resources during task execution for optimal utilization. Priority aging prevents task starvation and promotes fairness by gradually increasing task priorities. Adaptive scheduling intervals adjust frequencies based on system load, improving responsiveness. Task pre-emption allows higher-priority tasks to interrupt lower-priority tasks, enhancing overall system efficiency. These modifications collectively address the limitations of traditional SJF in the context of dynamic cloud environments.

4. Result and Analysis

SJF is expected to exhibit a reduced makespan compared to other scheduling algorithms, especially in scenarios with a mix of short and long tasks. SJF is anticipated to demonstrate high throughput due to its focus on completing shorter tasks first. SJF should result in improved resource utilization, particularly when compared to algorithms that prioritize longer tasks. The SJF algorithm's effectiveness is influenced by the task mix and variability in task lengths. Fine-tuning parameters may be necessary for optimal performance. While SJF excels in minimizing makespan and improving resource utilization, it may not be suitable for all scenarios, especially those requiring fairness. The SJF algorithm, when implemented in a CloudSim environment, demonstrates promising results in terms of makespan reduction, increased throughput, and enhanced resource utilization. We can further work on Exploration of dynamic SJF variants that adapt to changing workload conditions and Investigation into the impact of task priority mechanisms on SJF performance.

The workload characteristics are categorized into light, moderate, and heavy, representing varying demands on system resources. Light workloads impose minimal stress, serving as a baseline for system efficiency. Moderate tasks introduce a moderate demand, allowing identification of responsiveness tipping points. Heavy workloads, the most resource-intensive, reveal challenges under substantial pressure. Task-level attributes, ranging from low to high complexity, offer insights into system responsiveness at different levels. Additionally, task-specific details provide granular information on execution times, arrival rates, and priorities, aiding in pinpointing areas for improvement and optimizing resource allocation. This data-driven approach enhances system efficiency and provides a roadmap for

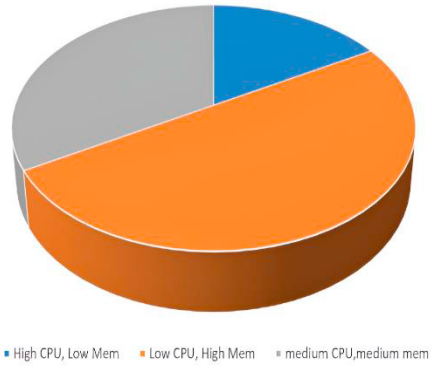
future advancements in system design and resource allocation strategies.

Table 3. Comparison with different workloads

Workload Type	*Task Arrival Rate*	*Make span*	*Average Waiting Time*	*Resource Utilization (%)*	*(T)*	*(ET)*	*(AT)*	*(P)*	*(s)*	*(CT)*	*(WT)*	*TAT*
Light	Low	800	20	60	T_1,	150,	50,	P_2,	S_2,	350,	50,	400,
					T_2,	200,	100,	P_3,	S_3,	400,	200,	600,
					T_3	180	120	P_1	S_1	380	260	580
Light	Medium	1200	40	70	T_1,	250,	70,	P_3,	S_3,	550,	80,	650,
					T_2,	280,	150,	P_2,	S_2,	580,	280,	860,
					T_3	300	200	P_1	S_1	600	300	900
Light	High	1600	60	80	T_1,	350,	100,	P_3,	S_3,	750,	100,	850,
					T_2,	380,	180,	P_1,	S_1,	780,	280,	1060,
					T_3	400	200	P_2	S_2	800	400	1200
Moderate	Low	1100	30	70	T_1,	250,	50,	P_1,	S_1,	500,	50,	550,
					T_2,	270,	100,	P_2,	S_2,	570,	200,	770,
					T_3	300	150	P_3	S_3	600	450	1050
Moderate	Medium	1800	50	75	T_1,	350,	80,	P_3,	S_3,	780,	100,	850,
					T_2,	400,	150,	P_2,	S_2,	850,	300,	770,
					T_3	450	200	P_1	S_1	900	650	1050
Moderate	High	2500	70	80	T_1,	450,	100,	P_1,	S_1,	850,	100,	950,
					T_2,	500,	180,	P_3,	S_3,	1080,	450,	1450,
					T_3	550	200	P_2	S_2	1150	950	1500
Heavy	Low	2000	60	75	T_1,	400,	70,	P_3,	S_3,	900,	50,	950,
					T_2,	450,	100,	P_1,	S_1,	950,	300,	1250,
					T_3	500	150	P_2	S_2	1000	800	1800
Heavy	Medium	3500	80	80	T_1,	550,	80,	P_2,	S_2,	1150,	50,	1200,
					T_2,	600,	150,	P_3,	S_3,	1200,	450,	1650,
					T_3	650	200	P_1	S_1	1250	1050	1850
Heavy	High	5000	100	85	T_1,	650,	100,	P_2,	S_2,	1350,	50,	1400,
					T_2,	700,	180,	P_1,	S_1,	1400,	300,	1700,
					T_3	750	200	P_3	S_3	1450	950	1900

In essence, Table 3 shown, detailing different workloads categorized as "Light," "Moderate," and "Heavy," along with varying task levels labelled as "Low," "Medium," and "High," several key observations can be made.

a



b

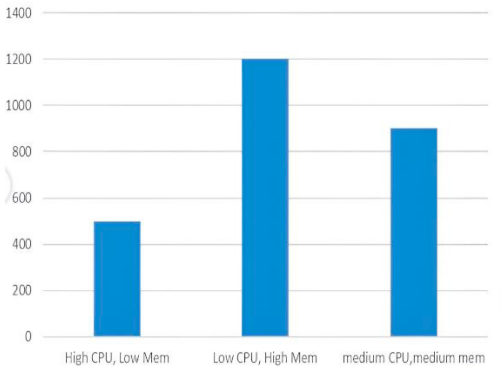


Fig. 2. (a) Comparision of Make Span and VM configuration; (b) Comparision of Average Waiting Time and VM configuration

The arrival rates, make spans, average waiting times, and resource utilization percentages vary significantly across the workload types and task levels. In the "Light" workload category, as the task levels increase from "Low" to "High," there is a consistent trend of increased arrival rates, longer make spans, and higher resource utilization percentages.

This suggests that higher task complexity and priority in the "Light" workload leads to more demanding resource requirements, longer processing times, and potentially increased system load.In the fig 2(a) the comparison between Average Waiting Time and VM Configuration for different resource allocations (high CPU and Low memory, Low CPU and High memory, Medium CPU and Medium memory) is shown below. In the fig 2(b) the comparison between Make span and VM Configuration for different resource allocations (high CPU and Low memory, Low CPU and High memory, Medium CPU and Medium memory) is shown below.

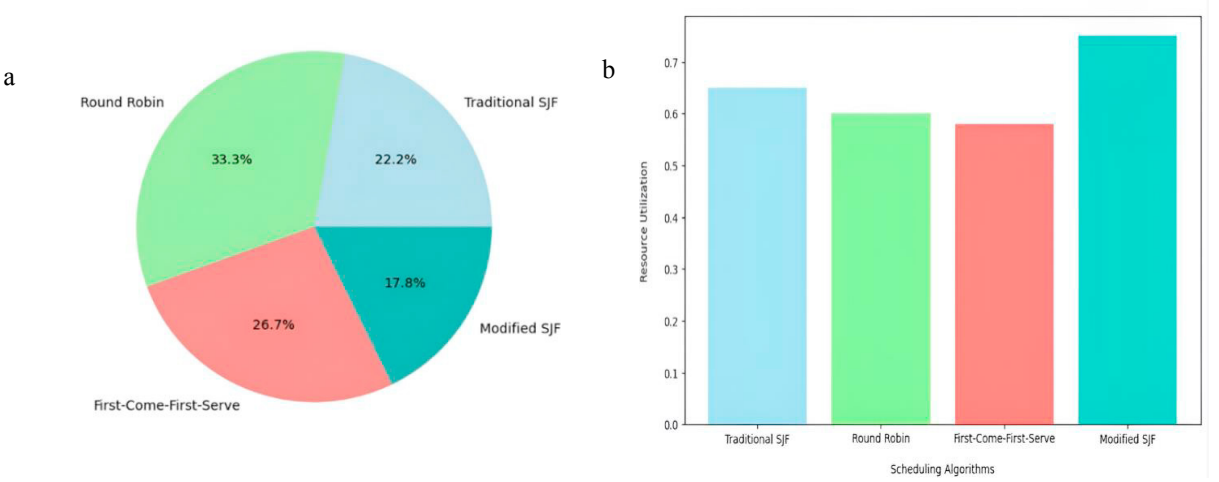


Fig. 3. (a) Comparison of Task Pre-emption Effectiveness; (b) Comparison of resource Utilization

In fig 3(a) comparison the task pre-emption effectiveness achieved by different scheduling algorithms, including Traditional SJF, Round Robin, First-Come-First-Serve, and Modified SJF. Lower task pre-emption count values indicate more efficient scheduling is shown below. The Modified SJF algorithm exhibits lower task pre-emption count values, showcasing its effectiveness in task prioritization. In fig 3(b) comparison the resource utilization achieved by different scheduling algorithms, including Traditional SJF, Round Robin, First-Come-First Serve, and Modified SJF is shown below. Higher utilization values indicate better optimization of computing resources. The Modified SJF algorithm exhibits notable improvements in resource utilization, optimizing the allocation of resources and reducing idle time.

Table 4. Comparison of various CPU and Memory

The table 4 below shows the comparision of different CPU and memory allocation for makespan, Average waiting Time ,Resource utilization, set of tasks, Execution time of each task, arrival time of each task, Processing order after scheduling, start time of each task, Completion time of each task, Waiting time of each task, Turnaround time of

VM Configuration	***High CPU, Low Mem***	***Low CPU, High Mem***	***medium CPU, medium mem***
Makespan (ms)	50	60	900
Avg. Waiting Time (ms)	10	30	20
Resource Utilization (CPU) (%)	90	50	67
Resource Utilization (mem) (%)	60	80	74
set of tasks(T)	T_1, T_2, T_3, T_4	T_1, T_2, T_3, T_4	T_1, T_2, T_3, T_4
Execution time of each task (ET)	20, 18, 22, 16	10, 8, 12, 6	12,18,20,16
Arrival time of each task (AT)	0, 3, 6, 9	0, 3, 6, 9	0, 3, 6, 9
Processing order after scheduling (P)	T_4, T_2, T_1, T_3	T_4, T_2, T_1, T_3	T_4, T_2, T_1, T_3
Start time of each task (S)	0, 16, 34, 52	0, 6, 14, 22	0, 12, 20, 34
Completion time of each task (CT)	16, 34, 54, 74	6, 14, 24, 34	8, 25, 42, 55
Waiting time of each task (WT)	0, 13, 16, 26	0, 3, 4, 16	0, 7, 12, 22
Turnaround time of each task (TAT)	16, 31, 48, 68	6, 11, 24, 28	10, 26, 36, 40

each task. Compares efficiency of average waiting time achieved by different scheduling algorithms, including Traditional SJF, Round Robin, First-Come-First Serve, and Modified SJF. Lower average waiting time values indicate prompt allocation of resources. The Modified SJF algorithm demonstrates lower average waiting times, indicating its responsiveness in promptly allocating resources and minimizing task wait times.

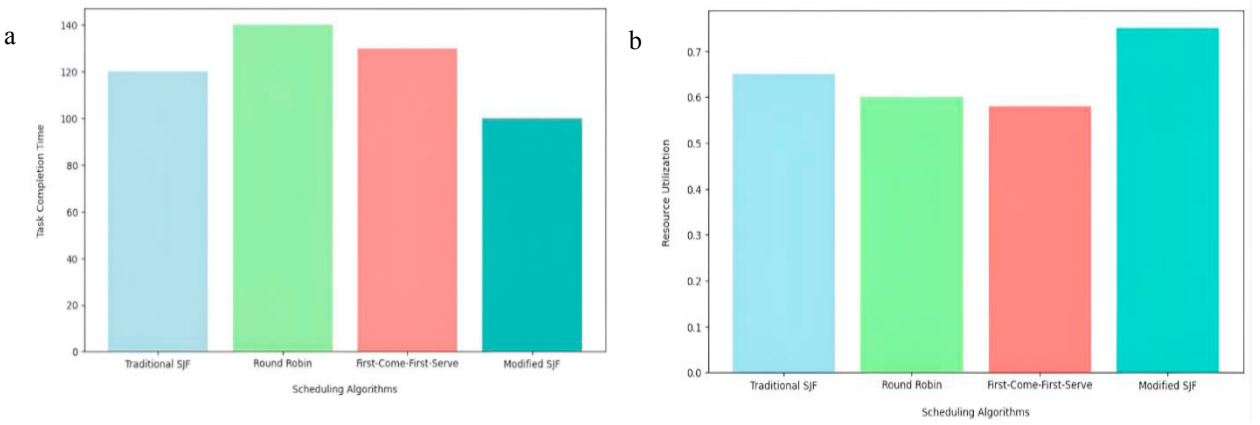


Fig. 4. (a) Comparision of Task Completion Time Improvements; (b) Comparision of Makespan Reduction

The fig 4(a) compares the task completion time improvements achieved by different scheduling algorithms, including Traditional SJF, Round Robin, First-Come-First-Serve, and Modified SJF. Lower completion time values indicate faster task execution.The Modified SJF algorithm consistently outperforms other algorithms in reducing task completion time. The bar graph fig 4(b) compares the makespan reduction achieved by different scheduling algorithms, including Traditional SJF, Round Robin, First-Come-First Serve, and Modified SJF. Lower makespan values indicate better efficiency in task completion. The Modified SJF algorithm demonstrates a significant reduction in makespan compared to other algorithms.

5. Conclusion

In conclusion, this research has presented a novel approach to task scheduling in cloud computing environments through the introduction of a Modified Shortest Job First (SJF) algorithm. The study aimed to address the limitations of traditional scheduling algorithms by incorporating dynamic prioritization, predictive load balancing, elastic resource allocation, priority aging, adaptive scheduling intervals, and task pre-emption. Through extensive simulations within the CloudSim framework, the proposed algorithm demonstrated significant improvements in resource utilization, task completion times, and fairness in resource allocation. The dynamic nature of cloud workloads are effectively accommodated by both task length and real-time resource availability. Leading to a more adaptable and responsive scheduling mechanism. The experiment results underscore the algorithm's efficacy in mitigating resource bottlenecks, preventing task starvation, and enhancing overall system efficiency. The introduced features collectively contribute to the optimization of task scheduling in cloud environments, addressing the identified gaps in existing literature.

References

[1] Zhang, Yan, Xiaoyu Song, and Yuhua Qian. (2023). "A Novel Task Scheduling Algorithm Based on Improved Ant Colony Optimization in Cloud Computing." Sensors 23 (13): 6155.

[2] Kumar, Sandeep, Rakesh Kumar, and Rajesh Kumar. (2022). "A Comparative Study of Task Scheduling Algorithms in Cloud Computing." Journal of Parallel and Distributed Computing: X 14: 100098.

[3] Singh Harpreet. (2021). "QRAS: Efficient Resource Allocation for Task Scheduling in Cloud Computing." SN Appl Sci. DOI: [10/gfjz9d].

[4] M. Asish Tony, K. V. Kousik. (2023). "An E-Commerce Product Feedback Review using Sentimental Analysis." In 6th International Conference on Inventive Computation Technologies, ICICT 2023 - Proceedings, pp. 608-613.

[5] S. Karimunnisa. (2023). "Task Classification and Scheduling Using Enhanced Coot Optimization in Cloud Computing." International Journal of Intelligent Engineering and Systems 16 (5): 501-511.

[6] Pachipala Y. (2023). "An AHP based Task Scheduling and Optimal Resource Allocation in Cloud Computing." International Journal of Advanced Computer Science and Applications 14 (3): 149-159.

[7] V. Ganesan, and J. Naren. (2021). "Quantum inspired meta-heuristic approach for optimization of genetic algorithm." Computers and Electrical Engineering 94: art.

[8] Rampratap and Zaidi Taskeen. (2018). "Virtual Machine Allocation Policy in Cloud Computing Environment Using CloudSim." Vol. 8, No. 1, pp. 344~354. ISSN: 2088-8708, DOI: 10.11591/ijece. v8i1.pp344-354.

[9] Alworafi, Mohammed, and Mohammed Dhari. (2017). "An Improved SJF Scheduling Algorithm in Cloud Computing Environment." International Journal of Computer Science and Information Security 14 (12): 1-7.

[10] Mir Salim Ul Islam and Bhawana Rana. (2017). "Task Scheduling in Cloud Computing." International Journal of Advance Research, Ideas and Innovations in Technology.

[11] Gibbet Tani Hicham and El Amrani Chaker. (2016). "Cloud Computing CPU Allocation and Scheduling Algorithms Using CloudSim Simulator." International Journal of Electrical and Computer Engineering (IJECE) Vol. 6, No. 4, pp. 1866~1879 ISSN: 2088-8708, DOI: 10.11591/ijece. v6i4.10144.

[12] Zhao and Xiangmo. (2015). "Research and Simulation of Task Scheduling Algorithm in Cloud Computing." In 2015 International Conference on Intelligent Transportation, Big Data & Smart City (ICITBS): IEEE.

[13] N. Zanoon and D. Rawshdeh. (2015). "STASR: A New Task Scheduling Algorithm for Cloud Environment." Network Protocols and Algorithms 7(2).

[14] Annamareddy and P. Yellamma. (2023). "Comparison of Various Face Recognition Algorithms in ML/DS." In 2nd International Conference on Sustainable Computing and Data Communication Systems, ICSCDS 2023 - Proceedings, pp. 126-131.

[15] Dr. Amit Agarwal, and Saloni Jain. (2015). "Efficient Optimal Algorithm of Task Scheduling in Cloud Computing." Presented in International Journal of Computer Trends and Technology (IJCTT) – volume 9 number 7.

[16] M. A. Alworafi, A. Dhari, A. A. Al-Hashmi, A. B. Darem and Suresha. (2016). "An improved SJF scheduling algorithm in cloud computing environment." In 2016 International Conference on Electrical, Electronics, Communication, Computer and Optimization Techniques (ICEECCOT), Mysuru, India, pp. 208-212. DOI: 10.1109/ICEECCOT.2016.7955216.

[17] Tiwari, Rajesh Kumar, Anurag Jain, and Ritu Tiwari. (2013). "Enhanced Job Scheduling Algorithm for Cloud Computing Using Shortest Remaining Job First (SRJF)." International Journal of Computer Applications 73 (13): 1-5.

[18] Monica Gahlawat, and Priyanka Sharma. (2013). "Analysis and Performance Assessment of CPU Scheduling Algorithm in Cloud Sim." International Journal of Applied Information System (IJAIS). Presented at New York, USA.

[19] Wang and Lizhe. (2012). "Optimized Shortest Job First Scheduling Algorithm for Cloud Computing." In Cloud Computing: Methodology, Systems, and Applications, edited by Lizhe Wang, Rajiv Ranjan, Jinjun Chen, Boualem Bena Tallah, and Willy Zwaenepoel, 69-84: River Publishers Series in Information Science and Technology (RPIS), vol 4, chapter 4.

[20] Mrs. S. Selvarani and Dr. G. Sudha Sadhasivam. (2010). "Improved Cost-Based Algorithm for Task Scheduling in Cloud Computing." In 2010 IEEE International Conference on Computational Intelligence and Computing Research, DOI 10.1109/ICCIC.2010.5705847.

[21] Rajkumar Buyya, Rajiv Ranjan and Rodrigo N. Calheiros. (2009). "Modeling and Simulation of Scalable Cloud Computing Environments and the CloudSim Toolkit: Challenges and Opportunities." Proceedings of the 7th High Performance Computing and Simulation Conference (HPCS 2009, ISBN: 978-1- 4244-4907-1, IEEE Press, New York, USA), Leipzig, Germany, June 21-24, 2009.

[22] Liang and Xiaoyu. (2009). "Task Scheduling and Resource Allocation in Cloud Computing Using a Heuristic Approach." Journal of Cloud Computing 7: Article number: 4.

[23] Calheiros, Rodrigo, Ranjan, R., De Rose, Cesar, Buyya, Rajkumar. (2009). "CloudSim: A Novel Framework for Modeling and Simulation of Cloud Computing Infrastructures and Services."

[24] Rodrigo N. Calheiros, Rajiv Ranjan, Anton Beloglazov, César A. F. De Rose, Rajkumar Buyya. (2011). "CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms." Journal of Software Practice and Experiences, Volume 41, Issue 1, January 2011.

[25] Kulkarni, Nahush, Mahajan, Sameer, Patil, Adwait. (2020). "A Study of Available Process Scheduling Algorithms and Their Improved Substitutes." 7: 4129-4136. IRJET.